TATA MOTORS
Connecting Aspirations

TATA

Active Workspace Client Customization/Configuration

Presented By – Sanjoy Mondal

Date – 14/09/2021

Contents – Configuration & Customization

Configuring page layout

1. Using XML rendering templates (XRT) with Active Workspace
2. Configuring Table

Changing Active Workspace behavior

1. Active Workspace development environment
2. Active Workspace development scripts
3. Active Workspace module
4. How to create boilerplate definitions

Examples:-

1. Add Theme
2. Create a Command
3. Add a visual indicator
4. Create a new Pages

Declarative user interface

What is the declarative UI?

A capability provided by the Active Workspace framework that allows for a concise and codeless definition of UI view content, layout, routing, and behavior. Actions, messages, service calls and their inputs and outputs can be mapped and described using **HTML** and **JSON**. It provides an *abstracted* means of defining the client UI and its behaviors; the underlying implementation is hidden.

Why a declarative UI?

There are many reasons to use an abstracted, declarative user interface. It provides increased:

1. Performance - Reducing the number of lines of code decreases load times.

2. Efficiency - Enables faster iteration and reduces the need to develop and maintain code. A simple declarative view definition allows UX specialists to author the desired UI, while working with an application developer to wire up the view model based on the intent.

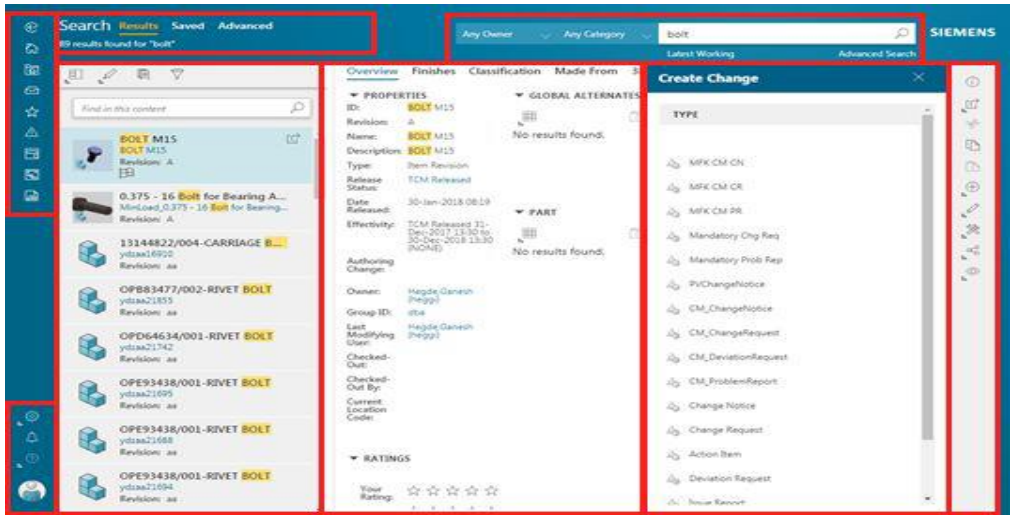
3. Sustainability - Since the Active Workspace declarative elements are abstracted, the underlying web technology can evolve with minimal to no effect on existing layouts.

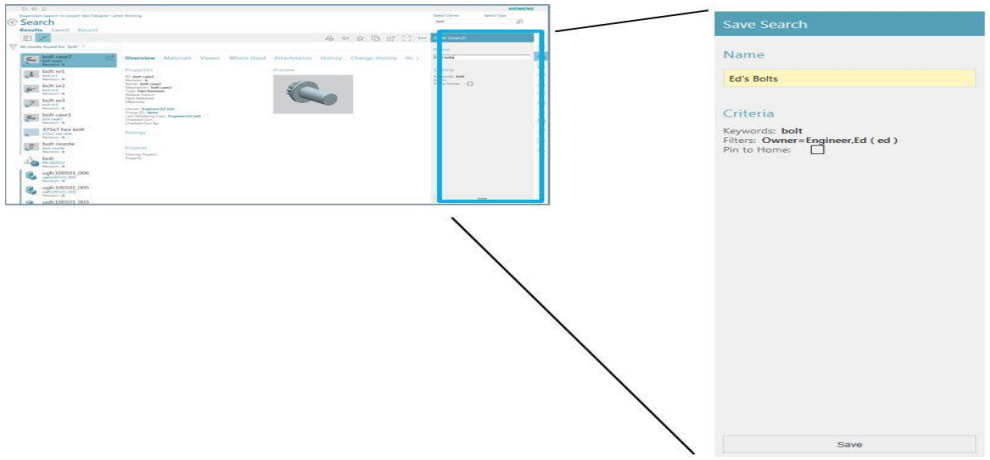
4. Consistency -Each page and panel element is built from basic, modular UI elements instead of custom pieces.

Declarative user interface

How does it work?

Declarative pages describe a single screen of the user experience and consist of several panels





A declarative page is the entire presentation in the browser window and uses a URL.

A declarative page is declaratively defined and shares the following characteristics:

- Follows layout and content patterns.
- Consists of several panels.
- Creates a view that is described using UI elements.
- Contains a view model that describes the page's data, i18n, and behaviors such as actions,conditions, messages, and routing.

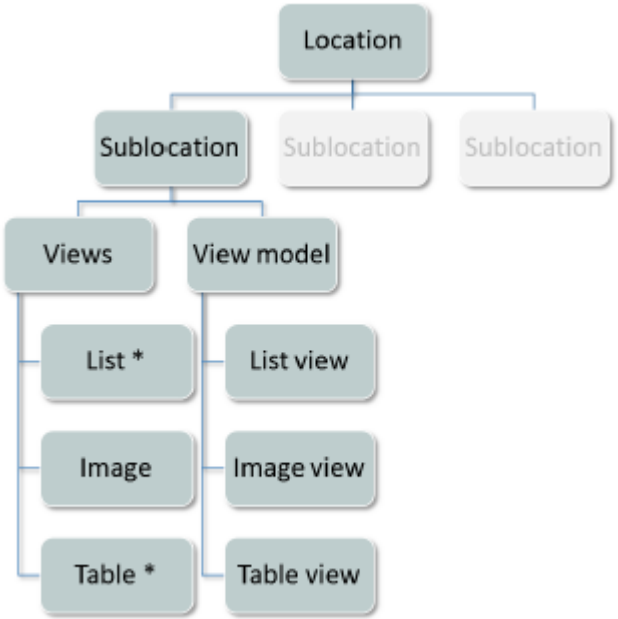
Declarative panels describe a region of the page.

Panels are also declaratively defined and share the following characteristics.

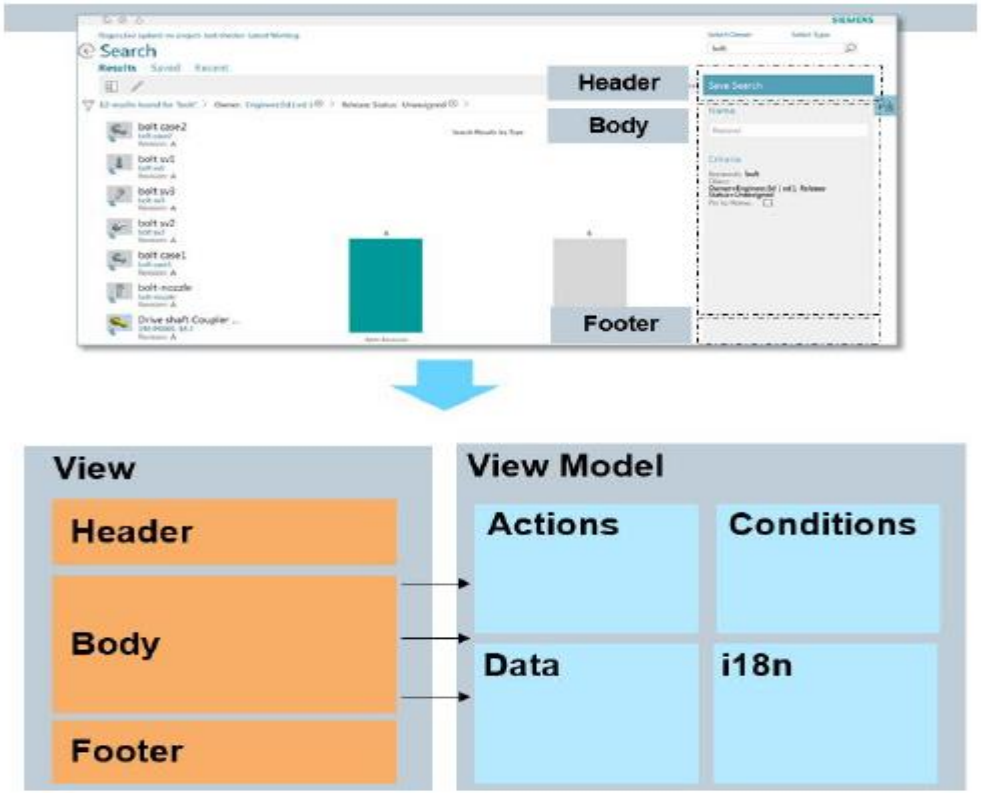
- Include a view that described using UI elements.
- Contain a view model that describes the panel's data, i18n, and behaviors such as actions,conditions, messages, and routing.

UI architecture

The Active Workspace UI consists mainly of sublocations grouped by locations. Each sublocation is defined by views, which rely on view models for their functionality.



The declarative definition of the UI has two main components, the view and the view model.

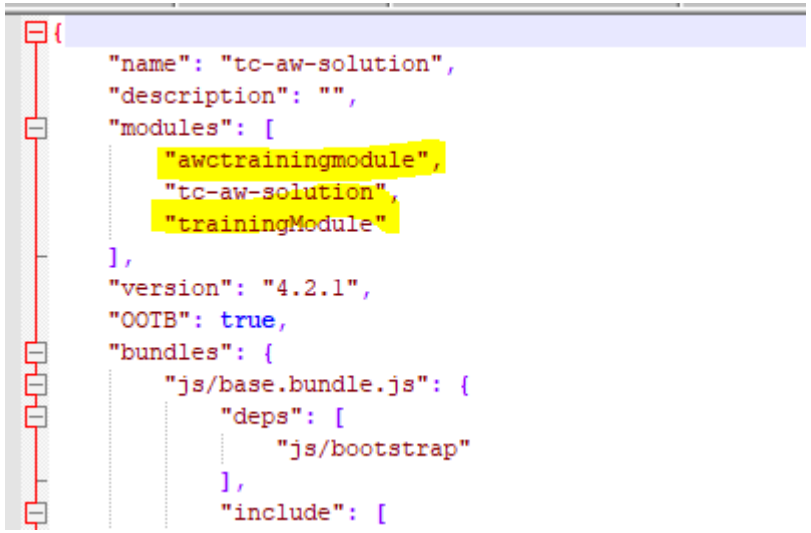


What files are involved?

The Active Workspace UI extends HTML with custom elements, which are part of **W3C**'s *Web Components*. Several custom elements work together to create content using the declarative UI:

- **Kit** A JSON text file that loads modules.
- **Module** A JSON text file that defines sublocations and commands.
- **View** A simple markup file (HTML) that controls a collection of related UI elements and their layout. These can be panels or pages, and may be displayed as the result of a command action or sublocation navigation. Views map to the view state which is defined in the view model.
- **View model** A JSON text file that is responsible for defining the view state, such as data, actions, i18n, and so on.
- **i18n** A JSON text file that provides localization capability for UI components.

1. kit.json



```
{
    "name": "tc-aw-solution",
    "description": "",
    "modules": [
        "awctrainingmodule",
        "tc-aw-solution",
        "trainingModule"
    ],
    "version": "4.2.1",
    "OOTB": true,
    "bundles": {
        "js/base.bundle.js": {
            "deps": [
                "js/bootstrap"
            ],
            "include": [
```

This singular file is provided OOTB, and is located in the **TC_ROOT\aws2\stage\src\solution** directory. It contains the definition for your installed configuration. You must register any modules you create in the **kit.json** file. You may edit this file manually, but using the **generateModule.cmd** script to create a module does this for you.

What files are involved?

2. module.json

A **module** may list other modules as dependents. In this file you define commands and their actions, conditions, and placement. Use the **generateModule** script to create this file initially. It creates the necessary directory structure and boilerplate files within the module directory

The **module.json** file contains the following information

```
{
    "name": "awctrainingmodule",
    "description": "This is the awctrainingmodule module",
    "skipTest": true
}
```

commandsViewModel

```
{
    "commands": {
    },
    "commandHandlers": {
    },
    "commandPlacements": {
    },
    "actions": {
    },
    "conditions": {
    },
    "i18n": {
    }
}
```

This is where you define your commands. You may instead use the **commandsViewModel.json** file to declare your command block. It must be a sibling of the **module.json** file.

What files are involved?

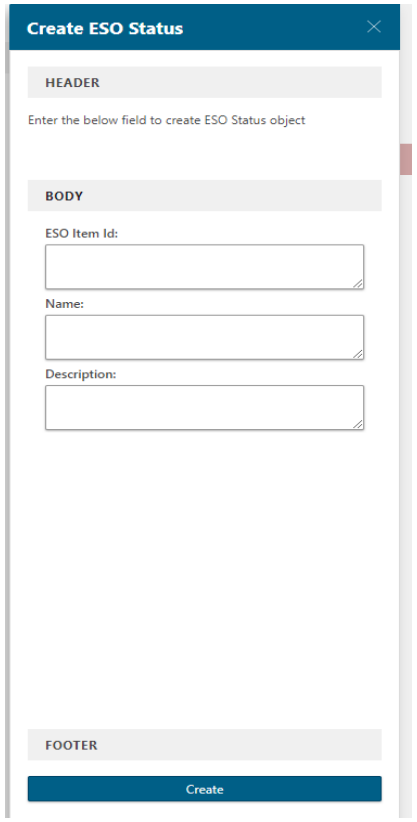
3. Declarative view

The view is an HTML markup file(…View.html) consisting of UI elements. The view is also responsible for:

- Defining the view hierarchy including sections and content.
- Mapping to data, actions, conditions, and i18n that are defined in the view model.
- Controlling the visibility of UI elements using **visibleWhen** clauses.

These files are located in the module\src\html directory.

```
<aw-command-panel caption="i18n.createESOTitle">
    <aw-panel-header>
        <aw-panel-section caption="i18n.header" >
            <p>Enter the below field to create ESO Status object</p>
        </aw-panel-section>
    </aw-panel-header>
    <aw-panel-body>
        <aw-panel-section caption="i18n.body">
            <div> <aw-widget prop="data.itemid"> </aw-widget> </div>
            <div> <aw-widget prop="data.name"> </aw-widget> </div>
            <div> <aw-widget prop="data.desc"> </aw-widget> </div>
        </aw-panel-section>
    </aw-panel-body>
    <aw-panel-footer>
        <aw-panel-section caption="i18n.footer">
            <aw-button action = "createObject">
                <aw-i18n>i18n.create</aw-i18n>
            </aw-button>
        </aw-panel-section>
    </aw-panel-footer>
</aw-command-panel>
```



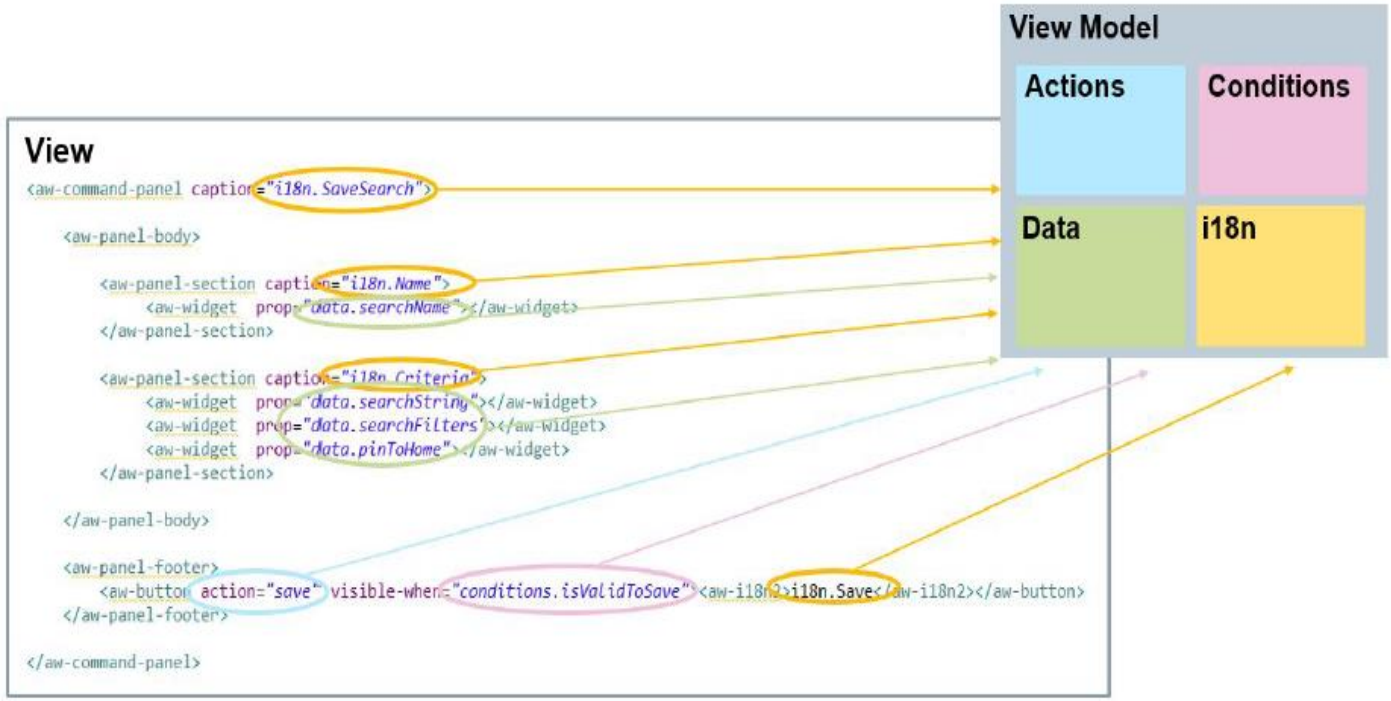
What files are involved?

4. Declarative view model

The view model is a JSON file (…ViewModel.jason). These files are located in the module\src\viewmodel directory.
It is responsible for view state such as:

- Imports
- Actions
- Data
- Conditions
- I18n

Mapping the view to the view model

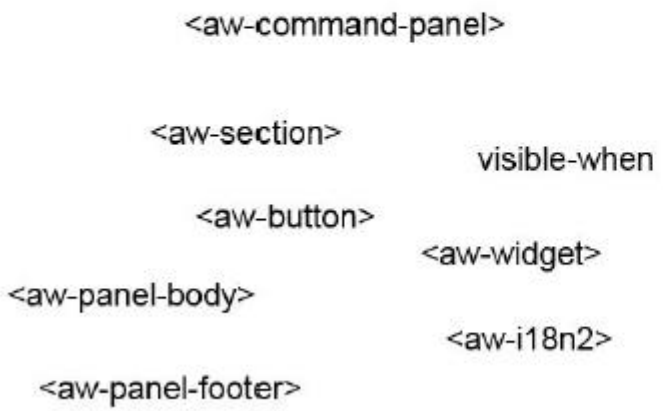


What files are involved?

4. Declarative view model

Imports

Imports are used to indicate the custom elements we want to use in our view and view model:



View Model

```
{
    "imports": [ "js/aw-command-panel.directive",
                 "js/aw-panel-body.directive",
                 "js/aw-panel-section.directive",
                 "js/aw-panel-footer.directive",
                 "js/aw-button.directive",
                 "js/aw-widget.directive",
                 "js/aw-i18n2.directive",
                 "js/visible-when.directive"],
```

What files are involved?

4. Declarative view model

Actions

The **actions** JSON object consists of the following components.

actionType
Supported options: TcSoaService, JSFunction, and RESTService

inputData
JSON data for the action input

outputData
JSON data for the action output

events
Triggered in response to the action

actionMessages
User messages and condition support

What files are involved?

4. Declarative view model

Data

The view can refer to data and view state in the view model data section.

Using {{ }} allows declarative binding to the live data.

e.g.

```
"propDisplayName": "{{i18n.searchString}}",
```

View

```
<aw-panel-section caption="i18n.Name">
    <aw-widget prop="data.searchName"></aw-widget>
</aw-panel-section>

<aw-panel-section caption="i18n.Criteria">
    <aw-widget prop="data.searchString"></aw-widget>
    <aw-widget prop="data.searchFilters"></aw-widget>
    <aw-widget prop="data.pinToHome"></aw-widget>
</aw-panel-section>
```

View Model

```
"data":
{
    "searchName":
    {
        "propDisplayName": "",
        "propType": "STRING",
        "propIsRequired": "true"
    },

    "searchString":
    {
        "propDisplayName": "{{i18n.searchString}}",
        "propType": "STRING",
        "propIsRequired": "false",
        "propIsEditable": "false",
        "propDbValue": "{{ctx.search.searchString}}",
        "propDisplayValue": "{{ctx.search.searchString}}"
    },

    "searchFilters":
    {
        "propDisplayName": "{{i18n.filterString}}",
        "propType": "STRING",
        "propIsRequired": "false",
        "propIsEditable": "false",
        "propDbValue": "{{ctx.search.filterString}}",
        "propDisplayValue": "{{ctx.search.filterString}}"
    },

    "pinToHome":
    {
        "propDisplayName": "{{i18n.pinSearch}}",
        "propType": "BOOLEAN",
        "propIsRequired": "false",
        "propLabelPosition": "PROPERTY_LABEL_AT_SIDE"
    }
},
```

What files are involved?

4. Declarative view model

Conditions

Conditions evaluate to true or false, may use boolean expressions (&&, ||, ==, !=), and may be used as follows:

- Use in **visibleWhen** clauses.
- Use for event handling.
- Refer to the data model state.
- Update the view model state.

View Model

```
"conditions":
{
    "isValidToSave":
    {
        "expression": "data.searchName.dbValue && data.searchName.dbValue!=''"
    }
},
```

View

```
<aw-panel-footer>
    <aw-button action="save" visible-when="conditions.isValidToSave"><aw-i18n2>i18n.Save</aw-i18n2></aw-button>
</aw-panel-footer>

</aw-command-panel>
```

What files are involved?

4. Declarative view model

i18n

User-facing text is localized. The view refers to localizations defined in the **i18n** section of the view model.

i18n text is provided as a JSON bundle.

View

```
<aw-panel-section caption="i18n.Name">
    <aw-widget prop="data.searchName"></aw-widget>
</aw-panel-section>

<aw-panel-section caption="i18n.Criteria">
    <aw-widget prop="data.searchString"></aw-widget>
    <aw-widget prop="data.searchFilters"></aw-widget>
    <aw-widget prop="data.pinToHome"></aw-widget>
</aw-panel-section>
```

View Model

```
"i18n":
{
    "Name":[ "SearchMessages","BaseMessage"],
    "SaveSearch": ["SearchMessages"],
    "Criteria": ["SearchMessages"],
    "Save":["SearchMessages"],
    "nameInUse": ["SearchMessages"],
    "CancelText": ["SearchMessages"],
    "OverwriteText": ["SearchMessages"],
    "pinToHome": ["SearchMessages"],
    "searchString": ["SearchMessages"],
    "filterString": ["SearchMessages"],
    "pinSearch": ["SearchMessages"]
}
```

What files are involved?

5. Declarative Messages

Messages are localized, are launched by actions, and cover information, warning, and error notifications. Messages may also::

- Present the user with options.
- Trigger actions.
- Leverage view model data and conditions.

Messages.json

This file is located in the **module\src\i18n** directory.

```
{
    "myGreenThemeTitle": "Green",
    "header": "Header",
    "body": "Body",
    "footer": "Footer",
    "checkBoxName": "Enable the OK button in footer",
    "save": "Ok",
    "textValue": "Contents here",
    "myCommandTitle": "Create ESO Status",
    "show": "Show",
    "create": "Create",
    "createESOTitle": "Create ESO Status",
    "showValues": "ESO Status \"{0}\" is created and Name is \"{1}\" and Description is \"{2}\"."
}
```

Examples – Create a new Command

Declarative commands require a native module. Its location is determined by which **uiAnchor** you use. In this case, the right wall command bar.

Step 1: Use the **generateModule** script to create the command. In this example, you create **myCommand**, choosing an existing icon and placing it on the right wall command bar.

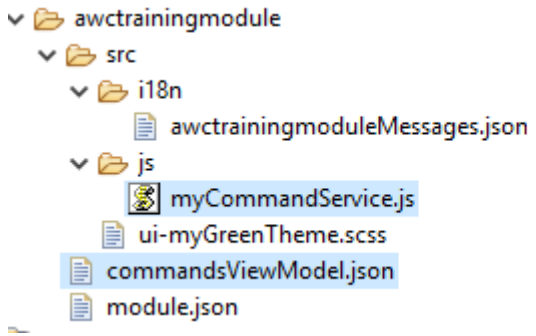
```
C:\Siemens\AwcTrng\TEAMCE~1\aws2\stage>generateModule
[Info] Select a type to generate. Available types are:

command: Generates a zero compile command.

kit: Generates a kit.json.
A kit.json is a combination of modules which will be included in the site build

location: Generates a location.
A location groups sublocations together within Active Workspace.
Examples are "Inbox" and "Search"

module: Generates a module.
A module is a build configuration that determines which code to include and how to process it.

state: Generates a state. A state is a page within a solution. The page's display is a declarative view and view model

subLocation: Generates a subLocation.
sublocation is a page which can be accessed in Active Workspace. The standard view consists of a primary and secondary w
orkarea.
Examples are "My Tasks" within "Inbox" location and "Results" within "Search" location

theme: Generates a theme.
A theme can change the colors that are used in your site. The default themes are Light and Dark

type: Generates a type.
This will allow you to define a custom type icon for a type.

workinstrView: Generates a zero compile workinstr gallery view within EWI solution.

workspace: Generates a workspace.

Enter type to generate: command
[14:42:34] info: Starting the command generator
[14:42:34] info: Selected module awctrainingmodule for modification.

Give the command a name. This is a unique identifier referenced by handlers and placements.
Command name: myCommand
```

Examples – Create a new Command


```
Number,cmdGenerateTransactions,cmdGeometric,cmdGeometricCrossSection,cmdGeometricMeasure,cmdGeometricQuery,cmdGeometricVolume,cmdGrantModificationRights,cmdGr
id,cmdGroupExpression,cmdGuide,cmdHalfFilledStar,cmdHideGraphOverview,cmdHighlight,cmdHome,cmdHomeLHN,cmdHorizontalSplit,cmdImage,cmdImage,cmdImport,cmdImport
Declaration,cmdImportExport,cmdImportExportExcel,cmdImportExportWord,cmdInbox,cmdIncludeVariants,cmdIncomingPartial,cmdIndexedModel,cmdInitiateSuspect,cmdInse
rt,cmdInsertImage,cmdInsertOLE,cmdJoinProccesses,cmdJoinProcesses,cmdLargeImageView,cmdLaunchSimulationTool,cmdLaunchWorkflow,cmdLayoutGroup,cmdLayoutHierarch
y,cmdLayoutSequence,cmdLicense,cmdLinktoNX,cmdListSummaryView,cmdListView,cmdMail,cmdManage,cmdManageDataCollectionSignatures,cmdManageDimensions,cmdManageWor
kPackage,cmdManualValidation,cmdMap,cmdMarkup,cmdMassAssign,cmdMatrixView,cmdMeasurementManagement,cmdMeasureTape,cmdMenu,cmdMergeBranches,cmdMissingImage,cmd
ModelViewPalette,cmdMore,cmdMoveDown,cmdMoveHere,cmdMoveTo,cmdMoveToLeftList,cmdMoveToRightList,cmdMoveToSection,cmdMoveUp,cmdMultipleCreate,cmdMultiSelect,cm
dNavigateAssociated,cmdNestedOn,cmdNestedOnOff,cmdNewActivity,cmdNewBuildElement,cmdNoteProperties,cmdOpen,cmdOpenInbox,cmdOpenInHostApplication,cmdOpenInNewT
ab,cmdOpenInNx,cmdOpenPackages,cmdOpenSharedEffectivity,cmdOppositeSelection,cmdOrientBack,cmdOrientBottom,cmdOrientFront,cmdOrientISO,cmdOrientLeft,cmdOrient
Right,cmdOrientTop,cmdOrientTrimetric,cmdOutgoingPartial,cmdOverideComplianceResult,cmdPack,cmdPan,cmdPanelBuilder,cmdParent,cmdParentPartial,cmdPaIcon name:
laneXy,cmdPlaneXz,cmdPlaneYz,cmdPlanNegativeXz,cmdPlay,cmdPMI,cmdPoint,cmdPorts,cmdPortsPartial,cmdPostEvent,cmdPrint,cmdProductionProgram,cmdPromote,cmdPromo
teTask,cmdPropagateSelection,cmdProximity,cmdProxyBid,cmdPublish,cmdQuickLink,cmdRadioSelected,cmdRadioUnselected,cmdRecipe,cmdRecordPartMovement,cmdRecordUti
lization,cmdRedo,cmdReflection,cmdRefresh,cmdRefreshFilter,cmdRegenerateBarcode,cmdReject,cmdRelatedObjects,cmdRelateProgramObjects,cmdRelease,cmdRemoteUpload
,cmdRemove,cmdRemoveAll,cmdRemoveAssignment,cmdRemoveAttribute,cmdRemoveFromStudy,cmdRemoveFromValidationContract,cmdRemoveLocation,cmdRemoveVariants,cmdRepla
te,cmdReplaceFile,cmdReplaceLatestRevision,cmdReply,cmdReports,cmdRequestSupplierDeclaration,cmdRescheduleDataRequirementItem,cmdReset,cmdResetForm,cmdResolve
PartMovement,cmdRetailImageTemplate,cmdReuseDocument,cmdRevertBranchToLatest,cmdRevertToDiscipline,cmdReviseToSolution,cmdRotate,cmdSave,cmdSaveAndExit,cmdSav
eAs,cmdSaveAsDiagram,cmdSaveCost,cmdSaveDiagram,cmdSaveSearch,cmdSaveWhatIf,cmdSaveWorkingContext,cmdSchedules,cmdScheduleTasks,cmdSearch,cmdSearchLatestData,
cmdSearchReport,cmdSelectFeatures,cmdSelectParts,cmdSendToSourcingManager,cmdSetActiveBranch,cmdSetAnchor,cmdSetAsContextSeason,cmdSetAsPredominantColor,cmdSe
tBaselineBranch,cmdSetLineage,cmdSetMeasurableAttributeInput,cmdSetMeasurableAttributeInputOutput,cmdSetMeasureableAttributeNone,cmdSetMeasureableAttributeOut
put,cmdSetPrimary,cmdSetPrimary,cmdSetStepStatus,cmdSetTemporaryNotifier,cmdSettings,cmdSettingsInProgress,cmdShadeMode,cmdShadow,cmdShapeSearch,cmdShapeSearc
hFilter,cmdShare,cmdShareCheckout,cmdShiftDate,cmdShow,cmdShowAggregatedPartsAndTools,cmdShowAllPorts,cmdShowAllResultsIn3dViewer,cmdShowAssociated,cmdShowAtt
achments,cmdShowCapsAndCutLines,cmdShowChildren,cmdShowCommentaryPanel,cmdShowConnectedPorts,cmdshowConnection,cmdShowDigitalAssets,cmdShowFiltered,cmdShowGra
phOverview,cmdShowGrid,cmdShowImpactOfChange,cmdShowIncomingRelations,cmdShowInfoPanel,cmdShowInterfaces,cmdShowMarkupPanel,cmdShowOnlyUpToDate,cmdShowOutgoin
gRelations,cmdShowParentBranch,cmdShowRelationsControls,cmdShowTable,cmdShowUntraced,cmdShowUnusedAttributes,cmdSignatureCertificate,cmdSignout,cmdSignOut,cmd
SimToolProgressMonitor,cmdSort,cmdSortAscending,cmdSortDecending,cmdSourcingManager,cmdStandIn,cmdStationPartMode,cmdStop,cmdStructureNavigate,cmdSubmitPartMo
vement,cmdSubmitToWorkflow,cmdSubstanceCategorize,cmdSurface,cmdSwap,cmdSystemSelection,cmdTableSummaryView,cmdTableView,cmdThumbsUp,cmdThumbsUpFilled,cmdTime
,cmdToggleBuildCondition,cmdToggleCompartment,cmdToggleLabel,cmdTraceLinkPartial,cmdTransfer,cmdTravelerReport,cmdTreeSummary,cmdTreeView,cmdTrihedron,cmdUndo
,cmdUngroupExpression,cmdUnlink,cmdUnlinkfromNX,cmdUnmap,cmdUnpack,cmdUnpin,cmdUnpostEvent,cmdUnsetLineage,cmdUnsubmitResponse,cmdUp,cmdUpdateAssignedRevision
,cmdUpdateDeliverableSequence,cmdUpdateDependantCosts,cmdUpdateSchedule,cmdUpload,cmdUpToDate,cmdUser,cmdUserAvailability,cmdUseTransparencyOff,cmdUseTranspar
encyOn,cmdValidate,cmdValidateVariantConfiguration,cmdValidateVariantExpression,cmdVariantOptions,cmdVertex,cmdVerticalSplit,cmdView,cmdViewConvenience,cmdVie
wFlat,cmdViewSingleBranch,cmdViewSingleLevel,cmdWalk,cmdWhatIfAnalysis,cmdWorkInProgressLimit,cmdWrapText,cmdXyPlanar,cmdXzPlanar,cmdYzPlanar,cmdZoom,cmdZxPla
nar

Icon name: cmdPlay

Choose a placement for this command.
This placement should match the anchor of the command bar you want this command to appear in.
Common placements are:
aw_rightWall
aw_oneStep
aw_primaryWorkArea
aw_3dViewer
aw_relationsViewer
aw_footer
aw_universalViewer
aw_toolsAndInfo
aw_ganttViewer
aw_graph_node
```

Examples – Create a new Command

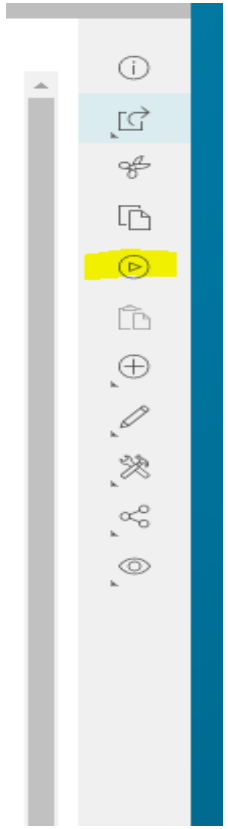

```
Icon name: cmdPlay

Choose a placement for this command.
This placement should match the anchor of the command bar you want this command to appear in.
Common placements are:
aw_rightWall
aw_oneStep
aw_primaryWorkArea
aw_3dViewer
aw_relationsViewer
aw_footer
aw_universalViewer
aw_toolsAndInfo
aw_ganttViewer
aw_graph_node

Placement name: aw_rightWall
[14:47:38] info: Created myCommandService.js
[14:47:38] info: Created command myCommand in module awctrainingmodule

C:\Siemens\AwcTrng\TEAMCE~1\aws2\stage>
```

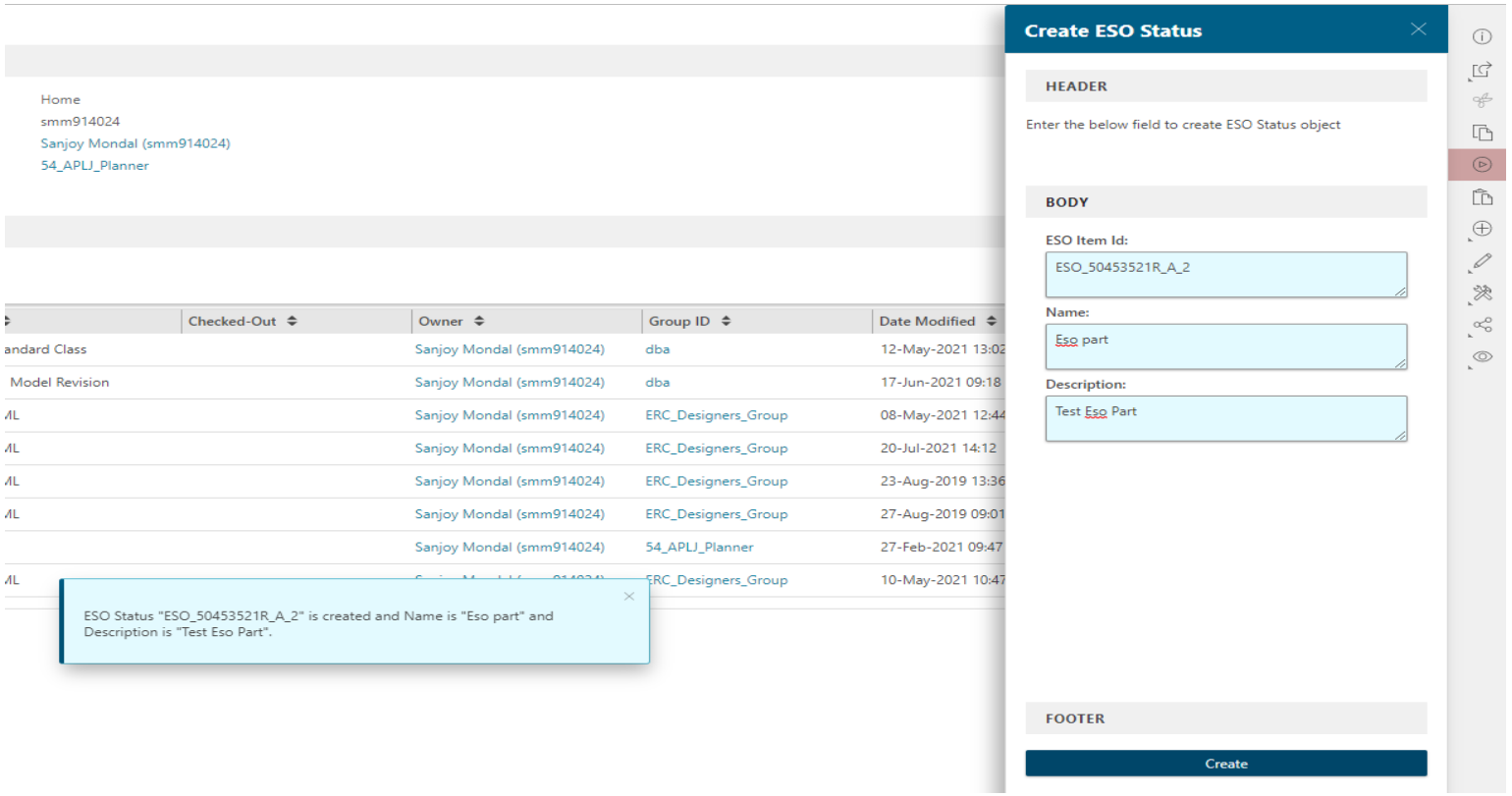


Examples – Create a new Command

Step 2: Create and implement your custom command View(myCommandView.html) and View model(eg myCommandViewModel.json)

Step3: Build the application, deploy, and test.



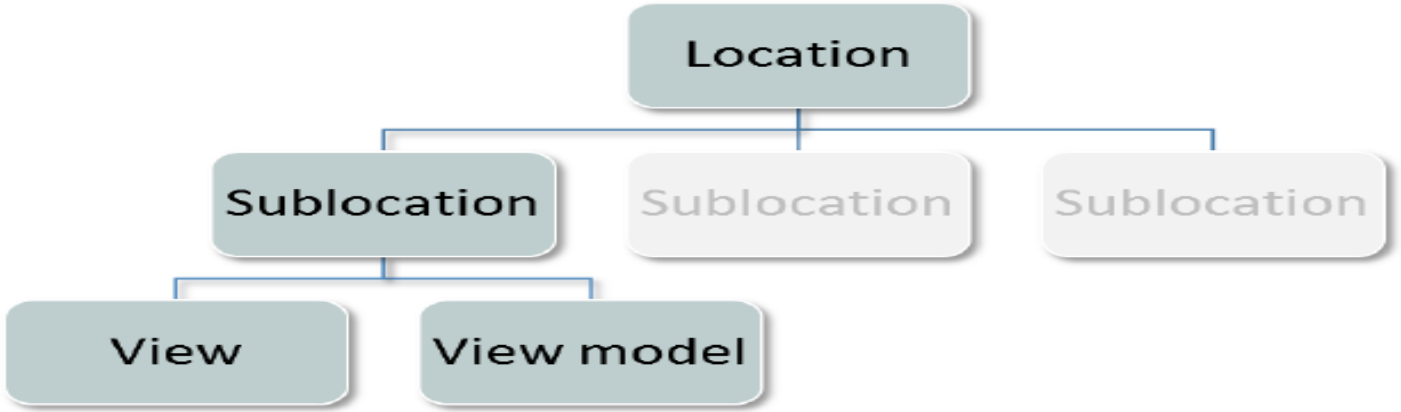


Examples – Create a Location/Sublocatiom

What is a sublocation?

A sublocation is a page in the UI that displays information and has a URL that points directly to it.. Every sublocation must be assigned to a location, which is a grouping of related sublocations. Any number of sublocations may be assigned to a location. If only a single sublocation exists for a location, then the sublocation name is not shown.

A sublocation is defined by *views* and *view models*.



Sublocation example

Examples of sublocations are: **My Tasks**, **Team**, **Tracking**, and so on. They are all part of the **Inbox** location.

The URL for the **My Tasks** sublocation is `http://host:port/#/myTasks`.

Examples – Create a Location/Sublocatiom

1. Create a Location

Step 1: C:\Siemens\AwcTrng\TEAMCE~1\aws2\stage>**generateModul**e

Step 2: Enter type to generate: **location**

Step3: [14:50:24] info: Starting the location generator
Multiple modules available: awctrainingmodule,trainingModule
Select a module to modify: **awctrainingmodule**

Step 4: Location name: **AwcTrainingLocation**
[14:51:23] info: Added location state AwcTrainingLocation to awctrainingmodule

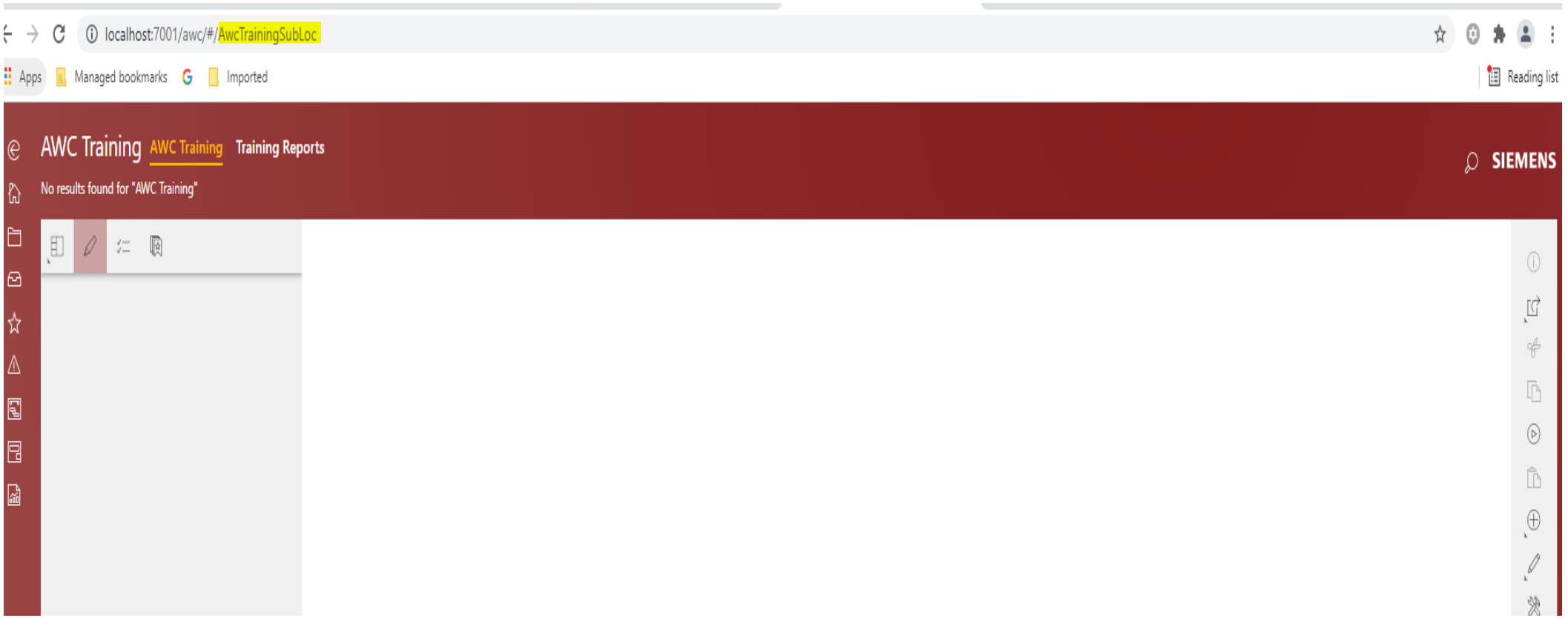
2. Create a Sub Location

Step 1: C:\Siemens\AwcTrng\TEAMCE~1\aws2\stage>**generateModul**e
Step2: Enter type to generate: **subLocation**
Step3: Location name: **AwcTrainingLocation**
Step 4: Sublocation name: **AwcTrainingSubLoc**

```
Give the sub-location a name. This is used as the URL and initial title of your sublocation.
Sublocation name: AwcTrainingSubLoc
[14:59:53] info: Created AwcTrainingSubLocImageView.html
[14:59:53] info: Created AwcTrainingSubLocListView.html
[14:59:53] info: Created AwcTrainingSubLocTableView.html
[14:59:53] info: Created AwcTrainingSubLocImageViewModel.json
[14:59:53] info: Created AwcTrainingSubLocListViewModel.json
[14:59:53] info: Created AwcTrainingSubLocTableViewModel.json
[14:59:53] info: Added new sub-location AwcTrainingSubLoc to location AwcTrainingLocation in awctrainingmodule
[14:59:53] info: You can access your new sublocation at http://localhost:3000/#/AwcTrainingSubLoc after rebuilding
```

Examples – Create a Location/Sublocatiom

3. Compile and deploy the war file and test the functionality



TATA MOTORS
Connecting Aspirations

TATA

THANK YOU